Python for Artificial Intelligence

Advanced OOP: Encapsulation, Relationships & Abstract Classes

Week 2, Day 5

Today's Agenda

- Hour 1: Encapsulation (public/protected/private) and nested classes
- Hour 2: Association, aggregation, composition, and abstract classes
- Hour 3: Data classes, keyword arguments, and end-to-end practice
- By the end: you can control what parts of a class are meant to be touched from outside, model real relationships between classes, and enforce that subclasses implement what they promise to

Hour 1

Encapsulation & Nested Classes

CONCEPT

What Is Encapsulation?

- Encapsulation means controlling which parts of an object are meant to be accessed directly from outside the class
- So far, every attribute you've written (self.name, self.balance) has been fully open - anyone can read or change it
- Some data should be more protected: an account balance shouldn't be changed by just typing acc.balance = -999
- Python does NOT enforce this with hard rules like Java or C++ do - it uses naming conventions everyone agrees to follow

CONCEPT

Three Levels: Public, Protected, Private

- Public (self.name): accessible from anywhere, no restriction - the default, what you've used all along
- Protected (self._name): single underscore means 'internal use, please don't touch from outside' - a convention only, still technically accessible
- Private (self.__name): double underscore triggers name mangling, which makes accidental outside access much harder
- None of these are enforced by Python the way private actually is in Java - they are signals of intent, respected by convention

EXAMPLE

Public vs. Protected

Both are technically accessible from outside; the underscore is a signal, not a lock.

```
class Student:
    def __init__(self, name, gpa):
        self.name = name       # public
        self._gpa = gpa        # protected (convention only)

s = Student("Ali", 3.8)
print(s.name)   # Ali - normal access
print(s._gpa)   # 3.8 - still works, but signals 'internal use'
```

Private Attributes & Name Mangling

SYNTAX

```
class ClassName:
    def __init__(self):
        self.__attribute = value
```

PARAMETERS

`self.__attribute` double underscore prefix; Python internally renames it to self._ClassName__attribute

YOU CAN

✓ Access it normally from inside the class's own methods

✓ Still reach it from outside if you really need to, via its mangled name (not recommended)

YOU CANNOT

✗ Access self.__attribute directly from outside the class - Python raises an AttributeError

✗ Rely on double underscore as true security - it's a naming trick, not encryption

Returns: not applicable; this is a naming convention, not a method

EXAMPLE

Private Attributes in Action

```
class Student:
    def __init__(self, name, gpa):
        self.name = name
        self.__gpa = gpa      # private

s = Student("Ali", 3.8)
print(s.name)      # Ali - fine
print(s.__gpa)     # AttributeError!

# Python actually renamed it internally:
print(s._Student__gpa)   # 3.8 - the mangled name still works
```

CONCEPT

Why Name Mangling Actually Matters

- This isn't just a naming quirk - it prevents a real class of bugs in inheritance
- Imagine a child class's __init__ accidentally reuses the same attribute name as the parent
- With a public attribute, the child silently overwrites the parent's value - a hard-to-spot bug
- With a private (double-underscore) attribute, each class gets its own separately-mangled copy, so this can't happen by accident

EXAMPLE

The Bug Name Mangling Prevents (Part 1 of 2)

Without mangling, SavingsAccount's __init__ accidentally wipes out the parent's balance.

```
# Without name mangling (bug risk)
class BankAccount:
    def __init__(self, balance):
        self.balance = balance   # public

class SavingsAccount(BankAccount):
    def __init__(self, balance):
        super().__init__(balance)
        self.balance = 0   # accidentally resets it!

acc = SavingsAccount(1000)
print(acc.balance)   # 0 - the 1000 is gone
```

EXAMPLE

The Bug Name Mangling Prevents (Part 2 of 2)

With mangling, each class gets its own separately-named copy, so the mistake above cannot happen.

```
# With name mangling (safe)
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance   # private, mangled to _BankAccount__balance

    def get_balance(self):
        return self.__balance

class SavingsAccount(BankAccount):
    def __init__(self, balance):
        super().__init__(balance)
        self.__balance = 0   # this is a DIFFERENT variable: _SavingsAccount__balance

acc = SavingsAccount(1000)
print(acc.get_balance())   # 1000 - the parent's copy is untouched
```

THINK & PRACTICE

Quick Check

You have class Wallet: def __init__(self, cash): self.__cash = cash. Why does wallet.__cash raise an error from outside the class, and what is its actual mangled name?

ANSWER

Quick Check: Answer

```
# __cash raises AttributeError from outside because Python
# renamed it internally the moment the class was defined.

# Its actual mangled name is:
wallet._Wallet__cash

# accessing wallet.__cash looks for an attribute that
# literally does not exist under that name anymore.
```

THINK & PRACTICE

Think & Practice (Harder)

Create a class Employee with a private attribute __salary set in __init__. Add a public method get_salary(self) that returns it, and a method give_raise(self, amount) that increases it. Confirm employee.__salary raises an error but employee.get_salary() works.

ANSWER

Think & Practice (Harder): Answer

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.__salary = salary

    def get_salary(self):
        return self.__salary

    def give_raise(self, amount):
        self.__salary += amount

e = Employee("Sara", 50000)
print(e.get_salary())   # 50000
e.give_raise(5000)
print(e.get_salary())   # 55000
# print(e.__salary)    # AttributeError
```

CONCEPT

Nested (Inner) Classes

- A class can be defined inside another class - this is a nested (or inner) class
- Used when one class only makes sense as part of another: a Department only exists inside a University
- Benefits: logical grouping, and avoiding cluttering the global namespace with a class nobody needs on its own
- You access it as Outer.Inner, and can create it independently of the outer class

Nested Class Syntax

SYNTAX

```
class Outer:
    def __init__(self, x):
        self.x = x

    class Inner:
        def __init__(self, y):
            self.y = y
```

PARAMETERS

`Outer.Inner` how you refer to and create the nested class from outside

YOU CAN

✓ Create the inner class independently: Outer.Inner(...)

✓ Access the outer class's class variables directly from inside the inner class

✓ Pass an outer object explicitly into an inner method if it needs outer instance data

YOU CANNOT

✗ Automatically access the outer instance's attributes from inside the inner class without being given the outer object

Returns: not applicable; this is a class definition pattern

EXAMPLE

A University with a Nested Department Class

```
class University:
    def __init__(self, name):
        self.name = name

    class Department:
        def __init__(self, dept_name):
            self.dept_name = dept_name

        def display(self):
            print("Department:", self.dept_name)

uni = University("FAST")
dept = University.Department("Computer Science")
dept.display()   # Department: Computer Science
```

THINK & PRACTICE

Quick Check

Create a nested class Engine inside a class Car. Engine's __init__ should take horsepower. Create an Engine object directly using Car.Engine(300) without ever creating a Car object.

ANSWER

Quick Check: Answer

```
class Car:
    class Engine:
        def __init__(self, horsepower):
            self.horsepower = horsepower

e = Car.Engine(300)
print(e.horsepower)   # 300
# no Car object was ever created - the nested class
# can be used completely independently
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

Hour 2

Class Relationships & Abstract Classes

CONCEPT

Beyond Inheritance: How Classes Relate

- Inheritance (Day 9) is one way classes relate: 'is a kind of'
- But most real relationships between classes are 'has a' or 'uses a', not 'is a'
- Three named patterns describe these relationships, in increasing order of how tightly connected the classes are: association, aggregation, composition
- None of these need new syntax - they're just names for patterns you could already write, that make design intent clear

CONCEPT

Association: 'Uses A'

- The loosest relationship: one class simply interacts with another, without owning it
- Example: a Teacher teaches a Student, but neither owns the other, and they exist completely independently
- Recognize it by: one method takes an object of another class as an argument, uses it, and that's it

EXAMPLE

Association

```
class Student:
    def __init__(self, name):
        self.name = name

class Teacher:
    def teach(self, student):
        print("Teaching", student.name)

s = Student("Ali")
t = Teacher()
t.teach(s)   # Teaching Ali
```

CONCEPT

Aggregation: 'Has A' (Weak Ownership)

- One class contains a reference to other objects, but those objects can exist on their own too
- Example: a Department has Teachers - but if the Department were deleted, the Teacher objects would still exist elsewhere
- Recognize it by: the objects are created OUTSIDE and passed IN to the containing class

EXAMPLE

Aggregation

Teachers are created independently, then handed to Department - Department doesn't own their existence.

```
class Teacher:
    def __init__(self, name):
        self.name = name

class Department:
    def __init__(self, name, teachers):
        self.name = name
        self.teachers = teachers   # list passed in from outside

t1 = Teacher("Ali")
t2 = Teacher("Sara")
dept = Department("CS", [t1, t2])   # teachers existed before dept
```

CONCEPT

Composition: 'Has A' (Strong Ownership)

- The tightest relationship: one object completely owns another
- If the parent object is destroyed, the contained object has no reason to exist either
- Example: a House has Rooms - a Room without a House doesn't make sense in this design
- Recognize it by: the contained object is created INSIDE the owner's own __init__, not passed in

EXAMPLE

Composition

Room is created inside House's own __init__ - House owns it completely.

```
class Room:
    def __init__(self, room_type):
        self.room_type = room_type

class House:
    def __init__(self):
        self.room = Room("Bedroom")   # created inside, not passed in

h = House()
print(h.room.room_type)   # Bedroom
```

THINK & PRACTICE

Quick Check

For each pair, say whether it's association, aggregation, or composition: (a) a Car has an Engine created inside its own __init__, (b) a Library holds a list of Book objects that were created before the Library existed, (c) a Doctor examines a Patient passed into a method.

ANSWER

Quick Check: Answer

```
# (a) Car -> Engine created inside __init__: COMPOSITION
#     (strong ownership - Engine has no purpose outside a Car)

# (b) Library holds Books created before it existed: AGGREGATION
#     (weak ownership - Books can exist without the Library)

# (c) Doctor examines a Patient passed into a method: ASSOCIATION
#     (loosest - just interacts, doesn't own or store it)
```

CONCEPT

A Quick Word on Decorators

- You're about to see @abstractmethod, a new piece of syntax: a decorator
- A decorator is a line starting with @, placed directly above a method or class, that changes how it behaves
- You don't need to know how to write your own decorators yet - just recognize the pattern: @something above a def or class turns on extra behavior
- You'll see two decorators today: @abstractmethod and @dataclass

CONCEPT

Abstract Classes: Enforcing a Contract

- An abstract class is a class that cannot be instantiated directly - it exists only to be inherited from
- It declares abstract methods: methods with no implementation, that every subclass MUST implement
- This combines inheritance with enforcement: Python will refuse to create an object of a subclass that forgot to implement a required method
- Useful when you want to guarantee every subclass supports certain behavior, without dictating how

Abstract Class Syntax

SYNTAX

```
from abc import ABC, abstractmethod

class ClassName(ABC):
    @abstractmethod
    def method_name(self):
        pass
```

PARAMETERS

ABC imported base class that marks a class as abstract

`@abstractmethod` decorator marking a method as required, with no implementation here

YOU CAN

✓ Give an abstract class regular, fully-implemented (concrete) methods too, alongside abstract ones

✓ Create objects of a concrete subclass once it implements every abstract method

✓ Have a class be abstract even with zero abstract methods, just by

YOU CANNOT

✗ Create an object of the abstract class itself - Python raises a TypeError

✗ Create an object of a subclass that hasn't implemented every abstract method

Return: not applicable; this is a class-design pattern enforced at object-creation time

EXAMPLE

An Abstract Vehicle Class (Part 1 of 2)

Every Vehicle subclass must implement start_engine(), or Python won't let you create it.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start_engine(self):
        pass

class Car(Vehicle):
    def start_engine(self):
        print("Car engine started")
```

EXAMPLE

An Abstract Vehicle Class (Part 2 of 2)

Add a second subclass, then call the shared method through a loop.

```
class Bike(Vehicle):
    def start_engine(self):
        print("Bike engine started")

vehicles = [Car(), Bike()]
for v in vehicles:
    v.start_engine()   # same method name, different behavior
```

EXAMPLE

What Happens If You Forget an Abstract Method

```
# Trying to create the abstract class itself:
v = Vehicle()
# TypeError: Can't instantiate abstract class Vehicle
# without an implementation for abstract method 'start_engine'

# A subclass that forgets to implement it:
class Boat(Vehicle):
    pass

b = Boat()
# TypeError: Can't instantiate abstract class Boat
# without an implementation for abstract method 'start_engine'
```

CONCEPT

Abstract Classes Can Have Concrete Methods Too

- An abstract class isn't 100% abstract - it can mix required (abstract) methods with regular (concrete) ones
- Concrete methods in the abstract class are inherited normally, just like any other parent method
- This lets you enforce only the parts that genuinely need to differ per subclass

EXAMPLE

Mixing Abstract and Concrete Methods

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start_engine(self):
        pass

    def honk(self):        # concrete - shared by all subclasses
        print("Beep beep!")

class Car(Vehicle):
    def start_engine(self):
        print("Car engine started")

c = Car()
c.honk()   # Beep beep! (inherited, no need to redefine)
```

THINK & PRACTICE

Quick Check

Create an abstract class Shape (using ABC) with an abstract method area(self). Create a concrete subclass Square(side) that implements area() as side ** 2. Then show what error you'd get if you tried Shape() directly.

ANSWER

Quick Check: Answer (Part 1 of 2)

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

sq = Square(4)
print(sq.area())   # 16
```

ANSWER

Quick Check: Answer (Part 2 of 2)

```
# shape = Shape()
# TypeError: Can't instantiate abstract class Shape
# without an implementation for abstract method 'area'
```

THINK & PRACTICE

Think & Practice (Harder)

Create an abstract class Payment with an abstract method pay(self, amount) and a concrete method receipt(self) that prints "Payment processed". Create a concrete subclass CardPayment that implements pay() to print a message including the amount. Demonstrate that both pay() and receipt() work on a CardPayment object.

ANSWER

Think & Practice (Harder): Answer

```
from abc import ABC, abstractmethod

class Payment(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

    def receipt(self):
        print("Payment processed")

class CardPayment(Payment):
    def pay(self, amount):
        print("Paid", amount, "by card")

p = CardPayment()
p.pay(500)       # Paid 500 by card
p.receipt()      # Payment processed
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

Hour 3

Data Classes, Keyword Arguments & Practice

CONCEPT

Data Classes: Less Boilerplate for Data-Only Classes

- Some classes exist purely to hold data with almost no behavior - just a bunch of attributes
- Writing __init__ by hand for every attribute gets repetitive for classes like this
- The @dataclass decorator generates __init__ (and a readable __str__-like display) for you automatically
- You'll see this pattern constantly later this course, when working with structured data from LLMs and APIs

@dataclass Syntax

SYNTAX

```
from dataclasses import dataclass

@dataclass
class ClassName:
    attribute1: type
    attribute2: type
```

PARAMETERS

`attribute1: type` attribute name followed by its expected type (e.g. name: str, price: float)

`@dataclass` decorator that auto-generates __init__ and a readable print() output

YOU CAN

✓ Create objects the normal way: ClassName(value1, value2)

✓ print() the object and get a readable summary automatically, with no __str__ needed

Returns: not applicable; this is a class-design shortcut

✓ Still add your own methods to a dataclass, just like a normal class

YOU CANNOT

✗ Skip the type after each attribute name - the syntax requires it, even though Python won't strictly enforce it at runtime

✗ Expect a dataclass to auto-generate custom methods beyond __init__ and display

EXAMPLE

A Product Class, With and Without @dataclass (Part 1 of 2)

First, here is the class written by hand.

```
# Without @dataclass
class Product:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity
```

EXAMPLE

A Product Class, With and Without @dataclass (Part 2 of 2)

Now the same class using @dataclass - same result, far less code.

```
# With @dataclass
from dataclasses import dataclass

@dataclass
class Product:
    name: str
    price: float
    quantity: int

p = Product("Oil Bottle", 450.0, 10)
print(p)
# Product(name='Oil Bottle', price=450.0, quantity=10)
```

CONCEPT

Keyword Arguments at Object Creation

- You already know keyword arguments from function calls (Day 6/7) - the same idea applies to creating objects
- Positional: values are matched to parameters by order
- Keyword: values are matched to parameters by name, in any order
- Keyword arguments make object creation far more readable once a class has several attributes

EXAMPLE

Positional vs. Keyword Object Creation

```
# Positional - depends on remembering the exact order
p1 = Product("Oil", 450, 10)

# Keyword - order doesn't matter, and it's self-documenting
p2 = Product(name="Oil", price=450, quantity=10)
p3 = Product(quantity=10, name="Oil", price=450)   # same result
```

CONCEPT

A Quick Note: Empty Classes

- class Student: pass - a class with nothing inside it at all
- Used as a placeholder, a marker, or a starting point for something you'll build out later
- You already saw this pattern in Day 9's Dog(Animal): pass, when a child class needed no changes at all

THINK & PRACTICE

Quick Check

Convert this manual class to a @dataclass: class Book: def __init__(self, title, author, price): self.title = title; self.author = author; self.price = price

ANSWER

Quick Check: Answer

```
from dataclasses import dataclass

@dataclass
class Book:
    title: str
    author: str
    price: float

b = Book("Dune", "Frank Herbert", 1200.0)
print(b)
# Book(title='Dune', author='Frank Herbert', price=1200.0)
```

WORKED EXAMPLE

Library System: Encapsulation + Composition + Abstract Class (Part 1 of 2)

First, the abstract item and a Book that implements it, keeping the author private.

```
from abc import ABC, abstractmethod

class LibraryItem(ABC):
    @abstractmethod
    def describe(self):
        pass

class Book(LibraryItem):
    def __init__(self, title, author):
        self.title = title
        self.__author = author   # private

    def describe(self):
        return self.title + " by " + self.__author
```

WORKED EXAMPLE

Library System: Encapsulation + Composition + Abstract Class (Part 2 of 2)

Now Library uses composition to hold a Book, and calls its describe() method.

```
class Library:
    def __init__(self, name):
        self.name = name
        self.catalog = Book("Dune", "Frank Herbert")   # composition

lib = Library("City Library")
print(lib.catalog.describe())   # Dune by Frank Herbert
```

THINK & PRACTICE

Practice (Harder)

Build a small system: an abstract class BankAccount with abstract methods deposit(amount) and withdraw(amount), and a concrete method display_balance(self) using a private __balance. Create a concrete subclass SavingsAccount that implements both abstract methods. Also create a @dataclass Customer with name, email, and account_id, and create one using keyword arguments.

ANSWER

Practice (Harder): Answer (Part 1 of 3)

```
from abc import ABC, abstractmethod
from dataclasses import dataclass

class BankAccount(ABC):
    def __init__(self, balance):
        self.__balance = balance

    @abstractmethod
    def deposit(self, amount):
        pass

    @abstractmethod
    def withdraw(self, amount):
        pass
```

ANSWER

Practice (Harder): Answer (Part 2 of 3)

```
    def display_balance(self):
        print("Balance:", self.__balance)

    def _change_balance(self, delta):   # protected helper
        self.__balance += delta

class SavingsAccount(BankAccount):
    def deposit(self, amount):
        self._change_balance(amount)

    def withdraw(self, amount):
        self._change_balance(-amount)

acc = SavingsAccount(1000)
acc.deposit(500)
acc.display_balance()   # Balance: 1500
```

ANSWER

Practice (Harder): Answer (Part 3 of 3)

```
@dataclass
class Customer:
    name: str
    email: str
    account_id: int

c = Customer(name="Ali", email="ali@example.com", account_id=101)
print(c)
```

Home Practice: Interview-Style Questions

- 1. What is encapsulation, and why doesn't Python enforce it as strictly as Java or C++?
- 2. What is name mangling, and what real bug does it help prevent in inheritance?
- 3. Give one example each of association, aggregation, and composition, and explain why each fits its category.
- 4. What makes a class abstract, and what happens if you try to instantiate one directly?
- 5. What does @dataclass generate for you automatically?
- 6. Rewrite a class of your choice from earlier in the course as a @dataclass.

Key Takeaways & Coming Up

- Encapsulation uses naming conventions (public, _protected, __private) to signal intent, not hard restriction
- Name mangling on __private attributes prevents accidental attribute clashes in inheritance
- Association, aggregation, and composition describe how tightly one class owns or uses another
- Abstract classes (ABC, @abstractmethod) enforce that every subclass implements required methods
- @dataclass removes boilerplate for classes that are mostly just structured data - a pattern you'll see again with LLM and API data
- Coming up in Day 11: Exception handling and file handling, writing code that survives errors and saves data